In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

# 1.1.1 Q1. Load the (SuperMarket Analysis.csv) dataset and display the first 5 rows.

In [4]:
```
path = '/Users/elzaniyazbekova/Desktop/ML/SuperMarket Analysis.csv'
s = pd.read_csv(path)
```

In [5]:
```
s.head(5)
```

Out[5]:

| | Invoice ID | Branch | City | Customer type | Gender | Product line | Unit price | Quantity | Ta |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 750-67-8428 | Alex | Yangon | Member | Female | Health and beauty | 74.69 | 7 | 26 |
| **1** | 226-31-3081 | Giza | Naypyitaw | Normal | Female | Electronic accessories | 15.28 | 5 | 3 |
| **2** | 631-41-3108 | Alex | Yangon | Normal | Female | Home and lifestyle | 46.33 | 7 | 16 |
| **3** | 123-19-1176 | Alex | Yangon | Member | Female | Health and beauty | 58.22 | 8 | 23 |
| **4** | 373-73-7910 | Alex | Yangon | Member | Female | Sports and travel | 86.31 | 7 | 30 |

# 1.1.2 Q2. Display the dataset shape (rows and columns).

In [6]:
```
s.shape
```

Out[6]:
```
(1000, 17)
```

# 1.1.3 Q3. List all column names.

In [7]:
```
s.columns
```

Out[7]:
```
Index(['Invoice ID', 'Branch', 'City', 'Customer type', 'Gender',
       'Product line', 'Unit price', 'Quantity', 'Tax 5%', 'Sales', 'Dat
e',
       'Time', 'Payment', 'cogs', 'gross margin percentage', 'gross inco
me',
       'Rating'],
      dtype='object')
```

# 1.1.4 Q4. Identify categorical and numerical columns.

categorical_cols = s.select_dtypes(include='object').columns numerical_cols = s.select_dtypes(exclude='object').columns print(categorical_cols) print(numerical_cols)

# 1.1.5 Q5. Check for missing values in each column.

In [10]:
```
s.isnull().sum()
```

Out[10]:
```
Invoice ID                 0
Branch                     0
City                       0
Customer type              0
Gender                     0
Product line               0
Unit price                 0
Quantity                   0
Tax 5%                     0
Sales                      0
Date                       0
Time                       0
Payment                    0
cogs                       0
gross margin percentage    0
gross income               0
Rating                     0
dtype: int64
```

# 1.1.6 Q6. Display the data types of each column.

In [11]:
```
s.dtypes
```

Out[11]:
```
Invoice ID                  object
Branch                      object
City                        object
Customer type               object
Gender                      object
Product line                object
Unit price                 float64
Quantity                     int64
Tax 5%                     float64
Sales                      float64
Date                        object
Time                        object
Payment                     object
cogs                       float64
gross margin percentage    float64
gross income               float64
Rating                     float64
dtype: object
```

# 1.1.7 Q7. Show summary statistics for numerical columns (use pandas method).

In [13]:
```
s.describe()
```

Out[13]:

| | Unit price | Quantity | Tax 5% | Sales | cogs | gross m percei |
|---|---|---|---|---|---|---|
| **count** | 1000.000000 | 1000.000000 | 1000.000000 | 1000.000000 | 1000.00000 | 1.000000 |
| **mean** | 55.672130 | 5.510000 | 15.379369 | 322.966749 | 307.58738 | 4.761905 |
| **std** | 26.494628 | 2.923431 | 11.708825 | 245.885335 | 234.17651 | 6.13149 |
| **min** | 10.080000 | 1.000000 | 0.508500 | 10.678500 | 10.17000 | 4.761905 |
| **25%** | 32.875000 | 3.000000 | 5.924875 | 124.422375 | 118.49750 | 4.761905 |
| **50%** | 55.230000 | 5.000000 | 12.088000 | 253.848000 | 241.76000 | 4.761905 |
| **75%** | 77.935000 | 8.000000 | 22.445250 | 471.350250 | 448.90500 | 4.761905 |
| **max** | 99.960000 | 10.000000 | 49.650000 | 1042.650000 | 993.00000 | 4.761905 |

# 1.1.8 Q8. Filter rows where Sales > 500.

In [24]:
```
s[s['Sales'] > 500]
```

Out[24]:

| | Invoice ID | Branch | City | Customer type | Gender | Product line | Unit price | Quantity |
|---|---|---|---|---|---|---|---|---|
| **0** | 750-67-8428 | Alex | Yangon | Member | Female | Health and beauty | 74.69 | 7 |
| **4** | 373-73-7910 | Alex | Yangon | Member | Female | Sports and travel | 86.31 | 7 |
| **5** | 699-14-3026 | Giza | Naypyitaw | Member | Female | Electronic accessories | 85.39 | 7 |
| **7** | 315-22-5665 | Giza | Naypyitaw | Member | Female | Home and lifestyle | 73.56 | 10 |
| **14** | 829-34-3910 | Alex | Yangon | Member | Female | Health and beauty | 71.38 | 10 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **988** | 267-62-7380 | Giza | Naypyitaw | Member | Male | Electronic accessories | 82.34 | 10 |
| **989** | 430-53-4718 | Cairo | Mandalay | Member | Male | Health and beauty | 75.37 | 8 |
| **991** | 602-16-6955 | Cairo | Mandalay | Normal | Female | Sports and travel | 76.60 | 10 |
| **996** | 303-96-2227 | Cairo | Mandalay | Normal | Female | Home and lifestyle | 97.38 | 10 |
| **999** | 849-09-3807 | Alex | Yangon | Member | Female | Fashion accessories | 88.34 | 7 |

227 rows × 17 columns

# 1.1.9 Q9. Filter sales in City = “Yangon” and Sales > 200.

In [29]:
```
s[(s['City'] == "Yangon") & (s['Sales'] > 200)]
```

Out[29]:

| | Invoice ID | Branch | City | Customer type | Gender | Product line | Unit price | Quantity | Ta |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 750-67-8428 | Alex | Yangon | Member | Female | Health and beauty | 74.69 | 7 | 2( |
| **2** | 631-41-3108 | Alex | Yangon | Normal | Female | Home and lifestyle | 46.33 | 7 | 16 |
| **3** | 123-19-1176 | Alex | Yangon | Member | Female | Health and beauty | 58.22 | 8 | 23 |
| **4** | 373-73-7910 | Alex | Yangon | Member | Female | Sports and travel | 86.31 | 7 | 30 |
| **6** | 355-53-5943 | Alex | Yangon | Member | Female | Electronic accessories | 68.84 | 6 | 20 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | |
| **976** | 221-25-5073 | Alex | Yangon | Normal | Female | Food and beverages | 74.66 | 4 | 14 |
| **981** | 809-46-1866 | Alex | Yangon | Normal | Male | Health and beauty | 58.15 | 4 | 11 |
| **982** | 139-32-4183 | Alex | Yangon | Member | Female | Sports and travel | 97.48 | 9 | 43 |
| **990** | 886-18-2897 | Alex | Yangon | Normal | Female | Food and beverages | 56.56 | 5 | 14 |
| **999** | 849-09-3807 | Alex | Yangon | Member | Female | Fashion accessories | 88.34 | 7 | 30 |

204 rows × 17 columns

# 1.1.10 Q10. Sort all orders by Sales in descending order.

In [30]:
```
s.sort_values( by="Sales", ascending=False)
```

Out[30]:

| | Invoice ID | Branch | City | Customer type | Gender | Product line | Unit price | Quantity |
|---|---|---|---|---|---|---|---|---|
| **350** | 860-79-0874 | Giza | Naypyitaw | Member | Female | Fashion accessories | 99.30 | 10 |
| **167** | 687-47-8271 | Alex | Yangon | Normal | Male | Fashion accessories | 98.98 | 10 |
| **557** | 283-26-5248 | Giza | Naypyitaw | Member | Female | Food and beverages | 98.52 | 10 |
| **699** | 751-41-9720 | Giza | Naypyitaw | Normal | Male | Home and lifestyle | 97.50 | 10 |
| **996** | 303-96-2227 | Cairo | Mandalay | Normal | Female | Home and lifestyle | 97.38 | 10 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **402** | 236-86-3015 | Giza | Naypyitaw | Member | Male | Home and lifestyle | 13.98 | 1 |
| **443** | 192-98-7397 | Giza | Naypyitaw | Normal | Male | Fashion accessories | 12.78 | 1 |
| **223** | 279-62-1445 | Giza | Naypyitaw | Member | Female | Fashion accessories | 12.54 | 1 |
| **629** | 308-39-1707 | Alex | Yangon | Normal | Female | Fashion accessories | 12.09 | 1 |
| **822** | 784-21-9238 | Giza | Naypyitaw | Member | Male | Sports and travel | 10.17 | 1 |

1000 rows × 17 columns

# 1.1.11 Q11. Sort by Date (ascending) and then Time (ascending).

In [34]:

```
s1 = s.sort_values(by="Date", ascending=True)
s2 = s1.sort_values(by="Time", ascending = True)
s2
```

Out[34]:

| | Invoice ID | Branch | City | Customer type | Gender | Product line | Unit price | Quantity |
|---|---|---|---|---|---|---|---|---|
| **293** | 316-68-6352 | Alex | Yangon | Member | Female | Food and beverages | 36.36 | 2 |
| **913** | 522-57-8364 | Alex | Yangon | Member | Male | Fashion accessories | 51.34 | 8 |
| **180** | 130-98-8941 | Giza | Naypyitaw | Normal | Male | Fashion accessories | 64.26 | 7 |
| **200** | 606-80-4905 | Giza | Naypyitaw | Member | Female | Sports and travel | 19.15 | 6 |
| **669** | 679-22-6530 | Cairo | Mandalay | Normal | Female | Sports and travel | 40.62 | 2 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **106** | 628-34-3388 | Giza | Naypyitaw | Member | Female | Fashion accessories | 27.38 | 6 |
| **267** | 571-94-0759 | Cairo | Mandalay | Member | Female | Food and beverages | 74.60 | 10 |
| **893** | 715-20-1673 | Cairo | Mandalay | Normal | Male | Electronic accessories | 28.38 | 5 |
| **136** | 700-81-1757 | Alex | Yangon | Normal | Female | Electronic accessories | 26.31 | 5 |
| **935** | 235-06-8510 | Giza | Naypyitaw | Member | Male | Home and lifestyle | 85.72 | 3 |

1000 rows × 17 columns

# 1.1.12 Q12. Sort by Unit price and Quantity.

In [36]:
```
s.sort_values(by=['Unit price', 'Quantity'])
```

Out[36]:

| | Invoice ID | Branch | City | Customer type | Gender | Product line | Unit price | Quantity |
|---|---|---|---|---|---|---|---|---|
| **944** | 333-23-2632 | Alex | Yangon | Member | Male | Health and beauty | 10.08 | 7 |
| **572** | 239-48-4278 | Alex | Yangon | Member | Male | Food and beverages | 10.13 | 7 |
| **784** | 516-77-6464 | Giza | Naypyitaw | Member | Female | Health and beauty | 10.16 | 5 |
| **822** | 784-21-9238 | Giza | Naypyitaw | Member | Male | Sports and travel | 10.17 | 1 |
| **881** | 115-38-7388 | Giza | Naypyitaw | Member | Female | Fashion accessories | 10.18 | 8 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **283** | 667-92-0055 | Alex | Yangon | Member | Male | Health and beauty | 99.83 | 6 |
| **494** | 437-53-3084 | Cairo | Mandalay | Normal | Male | Fashion accessories | 99.89 | 2 |
| **930** | 641-62-7288 | Cairo | Mandalay | Normal | Male | Home and lifestyle | 99.92 | 6 |
| **983** | 148-41-7930 | Giza | Naypyitaw | Normal | Male | Health and beauty | 99.96 | 7 |
| **122** | 219-22-9386 | Cairo | Mandalay | Member | Female | Sports and travel | 99.96 | 9 |

1000 rows × 17 columns

# 1.1.13 Q13. Calculate the total sales in "Sales" per Branch.

In [40]:
```
s.groupby('Branch')['Sales'].sum()
```

Out[40]:
```
Branch
Alex     106200.3705
Cairo    106197.6720
Giza     110568.7065
Name: Sales, dtype: float64
```

## 1.1.14 Q14. Calculate average Sales per City.

In [42]:
```
s.groupby('City')['Sales'].mean()
```

Out[42]:
```
City
Mandalay     319.872506
Naypyitaw    337.099715
Yangon       312.354031
Name: Sales, dtype: float64
```

## 1.1.15 Q15. Find the quantities sold per product line.

In [47]:
```
s.groupby('Product line')['Quantity'].sum()
```

Out[47]:
```
Product line
Electronic accessories    971
Fashion accessories       902
Food and beverages        952
Health and beauty         854
Home and lifestyle        911
Sports and travel         920
Name: Quantity, dtype: int64
```

## 1.1.16 Q16. Calculate average gross income per Gender.

In [48]:
```
s.groupby('Gender')['gross income'].mean()
```

Out[48]:
```
Gender
Female    16.234829
Male      14.240749
Name: gross income, dtype: float64
```

## 1.1.17 Q17. Count number of sales per Payment method.

In [49]:
```
s.groupby('Payment')['Sales'].count()
```

Out[49]:
```
Payment
Cash           344
Credit card    311
Ewallet        345
Name: Sales, dtype: int64
```

# 1.1.18 Q18. Find maximum Sales per Branch.

In [50]:
```
s.groupby('Branch')['Sales'].max()
```

Out[50]:
```
Branch
Alex     1039.29
Cairo    1022.49
Giza     1042.65
Name: Sales, dtype: float64
```

# 1.1.19 Q19. Find minimum Unit price per Product line.

In [51]:
```
s.groupby('Product line')['Unit price'].min()
```

Out[51]:
```
Product line
Electronic accessories    10.56
Fashion accessories       10.18
Food and beverages        10.13
Health and beauty         10.08
Home and lifestyle        10.53
Sports and travel         10.17
Name: Unit price, dtype: float64
```

# 1.1.20 Q20. Find the sum of gross income per Product line and Branch.

In [54]:
```
s.groupby(['Product line', 'Branch'])['gross income'].sum()
```

Out[54]:
```
Product line            Branch
Electronic accessories  Alex       872.2435
                        Cairo      811.9735
                        Giza       903.2845
Fashion accessories     Alex       777.7385
                        Cairo      781.5865
                        Giza      1026.6700
Food and beverages      Alex       817.2905
                        Cairo      724.5185
                        Giza      1131.7550
Health and beauty       Alex       599.8930
                        Cairo      951.4600
                        Giza       791.2060
Home and lifestyle      Alex      1067.4855
                        Cairo      835.6745
                        Giza       661.6930
Sports and travel       Alex       922.5095
                        Cairo      951.8190
                        Giza       750.5680
Name: gross income, dtype: float64
```

### 1.1.21 21. What is the total quantities sold in Product line: “Electronic accessories”?

In [64]:
```
s[s['Product line'].str.contains('Electronic accessories')]['Quantity'].s
```

Out[64]:
```
np.int64(971)
```

### 1.1.22 22. What is the average Sales for female customers?

In [65]:
```
s[s['Gender'].str.contains('Female')]['Sales'].mean()
```

Out[65]:
```
np.float64(340.9314141856392)
```

### 1.1.23 23. What is the most expensive Unit price among Customer type members only?

In [77]:
```
s.groupby('Customer type')['Unit price'].max().head(1).item()
```

Out[77]:
```
99.96
```

### 1.1.24 24. How many orders with Rating >= 9 ?

In [83]:
```
(s['Rating'] >= 9).sum()
```

Out[83]:
```
np.int64(166)
```

### 1.1.25 25. What is the total Sales for Payment “Credit card” in Branch C?

In [93]:
```
s[s['Payment'].str.contains('Credit card') & (s['Branch'] == 'C')]['Sales
```

Out[93]:
```
np.float64(0.0)
```